

ATIVIDADE ACADÊMICA

CLOUD COMPUTING

Aplicação de Processamento Paralelo e Distribuído em Nuvem: Cálculo de Pi por Monte Carlo com Celery, Redis e Docker

Disciplina:	Cloud Computing
Tema escolhido:	Simulador de Tarefas Científicas
Stack:	Python, FastAPI, Streamlit, Celery, Redis, Docker e Docker Compose
Integrantes:	Nome 1; Nome 2; Nome 3; Nome 4
Data:	08/06/2026

Resumo

Este relatório apresenta o desenvolvimento de uma aplicação acadêmica voltada à demonstração prática de processamento paralelo, processamento distribuído e computação em nuvem. O escopo escolhido foi um simulador de tarefas científicas, baseado no cálculo aproximado do número Pi pelo método de Monte Carlo. A escolha desse problema se justifica por sua natureza computacionalmente intensiva, pois a precisão do resultado depende da execução de um grande número de iterações independentes.

A solução foi estruturada com uma interface web em Streamlit, uma API em FastAPI, um broker Redis e múltiplos workers Celery executados em containers Docker isolados. A carga de trabalho é particionada em blocos independentes, permitindo que diferentes workers processem simultaneamente partes do problema. Ao final, a API consolida os resultados parciais, calcula a estimativa de Pi e retorna ao usuário o tempo de execução total.

Os testes de desempenho compararam a execução com 1 worker e com múltiplos workers. Para 12.000.000 de iterações, a execução com 4 workers reduziu o tempo de 4,359 segundos para 1,691 segundo, evidenciando ganho de desempenho por paralelismo. O trabalho também apresenta a arquitetura, a lógica de distribuição, instruções de execução e orientações para deploy em nuvem.

1. Introdução

A computação em nuvem permite disponibilizar aplicações e recursos computacionais sob demanda, com escalabilidade, elasticidade e acesso remoto. Em cenários de processamento intensivo, a nuvem se torna relevante porque permite aumentar a capacidade de processamento sem depender apenas de uma máquina local. Esse princípio é aplicado em áreas como análise de dados, processamento de imagens, inteligência artificial, simulações científicas e mineração de grandes volumes de informação.

O processamento paralelo consiste em dividir uma tarefa em partes menores que podem ser executadas simultaneamente. Já o processamento distribuído amplia esse conceito ao permitir que essas partes sejam executadas por diferentes processos, containers ou nós, comunicando-se por meio de uma infraestrutura de coordenação. Neste projeto, a distribuição foi simulada com múltiplos containers Docker, cada um executando um worker Celery independente.

O objetivo geral da aplicação é demonstrar, de forma prática e mensurável, que uma carga computacional intensiva pode ser dividida, distribuída e executada com melhor desempenho quando comparada à execução sequencial ou com apenas um worker.

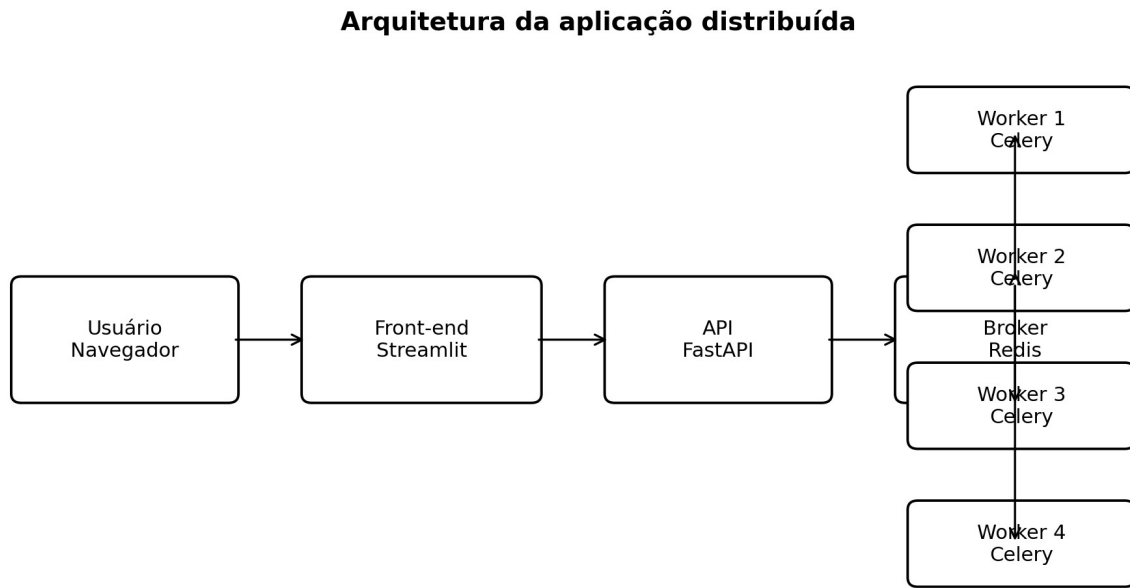
2. Tema escolhido e justificativa técnica

O tema selecionado foi Simulador de Tarefas Científicas, utilizando o cálculo de Pi por Monte Carlo. O método consiste em gerar pontos aleatórios dentro de um quadrado unitário e verificar quantos desses pontos caem dentro de um quarto de círculo. A razão entre os pontos internos e o total de pontos gerados permite estimar Pi pela expressão: $\text{Pi} \approx 4 \times \text{pontos internos} / \text{pontos totais}$.

A principal vantagem desse algoritmo para o trabalho acadêmico é que cada ponto pode ser avaliado de forma independente. Portanto, a tarefa pode ser naturalmente dividida em blocos, sem dependência entre os workers durante o processamento. Essa característica reduz a necessidade de comunicação entre processos e torna o problema adequado para paralelismo e distribuição.

Além disso, a quantidade de iterações pode ser aumentada facilmente, elevando a carga computacional e permitindo observar diferenças de desempenho entre execuções com 1 worker e com 4 ou mais workers.

Figura 1 - Diagrama básico da arquitetura utilizada



A carga é dividida em chunks independentes e processada pelos workers em containers isolados.

3. Arquitetura da solução

A aplicação foi organizada em camadas para separar responsabilidades. O usuário acessa a interface Streamlit pelo navegador, informa a quantidade de iterações e escolhe a quantidade de workers desejada. O front-end envia a requisição para a API FastAPI, responsável por validar os parâmetros, dividir a carga em blocos e encaminhar as tarefas ao broker Redis.

O Redis atua como intermediador de mensagens. Ele recebe as tarefas publicadas pela API e as disponibiliza aos workers Celery. Cada worker é executado em um container isolado, consumindo uma tarefa de cada vez e processando uma fração do cálculo. Ao finalizar, cada worker devolve seu resultado parcial para o backend, que consolida os valores e calcula a estimativa final de Pi.

A containerização foi adotada para simular uma infraestrutura distribuída. Mesmo quando executada localmente, a topologia representa uma arquitetura de nuvem composta por serviços independentes: front-end, API, broker e workers. Essa separação facilita o deploy posterior em plataformas como Railway, Render, Fly.io ou provedores compatíveis com Docker.

4. Tecnologias utilizadas

Camada	Tecnologia	Finalidade
Interface / Front-end	Streamlit	Página web simples para envio dos parâmetros e exibição do resultado final.
Backend / API	FastAPI	Recebe as requisições, valida os dados e coordena a execução distribuída.
Broker de mensagens	Redis	Fila responsável por intermediar a comunicação entre API e workers.
Workers distribuídos	Celery	Executa as tarefas em

		processos/containers independentes.
Containerização	Docker e Docker Compose	Isola os serviços e simula uma topologia distribuída multi-container.
Linguagem	Python	Implementação do algoritmo, API e integração entre camadas.

5. Lógica de paralelismo e distribuição

A lógica de paralelismo foi implementada pela divisão do total de iterações em chunks. Se o usuário solicita 12.000.000 de iterações e seleciona 4 workers, a API divide a carga em quatro blocos de aproximadamente 3.000.000 de iterações. Cada bloco é enviado como uma tarefa independente para o broker Redis.

Os workers Celery consomem as tarefas disponíveis na fila. Cada worker executa o mesmo algoritmo, porém com uma semente aleatória diferente, evitando repetição dos pontos gerados. Ao final de cada tarefa, o worker retorna apenas a quantidade de pontos que caíram dentro do círculo. Essa estratégia reduz o tráfego de dados, pois não é necessário retornar todos os pontos simulados, apenas o resultado agregado.

A distribuição ocorre porque os workers são serviços independentes definidos no Docker Compose. Eles não compartilham o mesmo processo de execução e podem ser escalados horizontalmente. Em um ambiente de nuvem, a mesma estrutura poderia ser distribuída entre diferentes instâncias ou réplicas, mantendo o Redis como camada de coordenação.

Pseudocódigo simplificado da execução:

1. Receber total de iterações e número de workers.
2. Dividir o total de iterações em N chunks.
3. Enviar cada chunk para o Redis como tarefa Celery.
4. Cada worker processa seu chunk independentemente.
5. A API soma os resultados parciais.
6. Calcular $Pi = 4 \times \text{pontos internos} / \text{pontos totais}$.
7. Retornar resultado, tempo total e configuração utilizada.

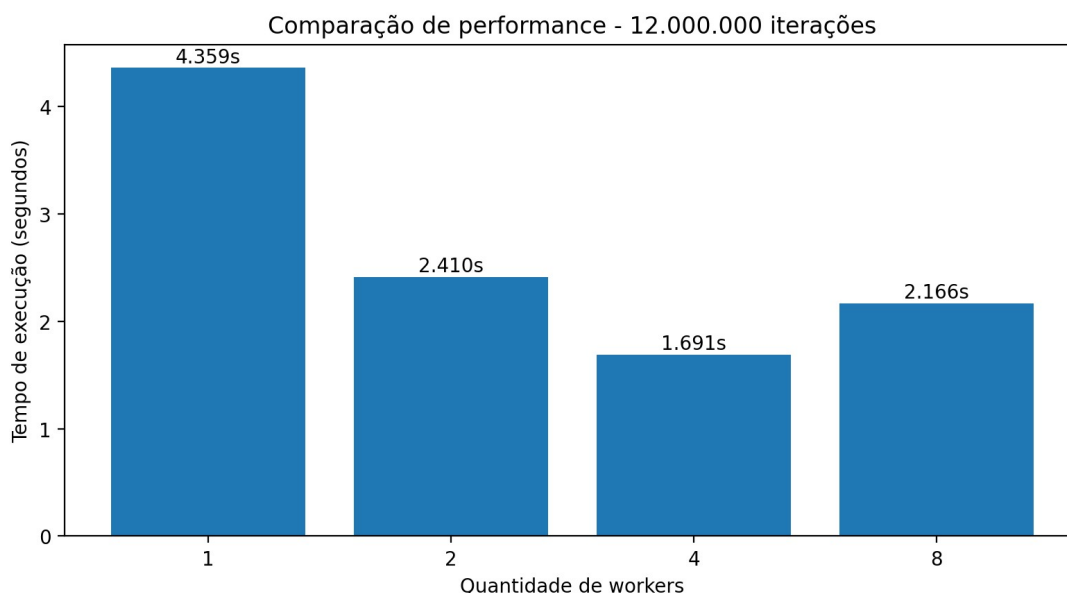
6. Validação de performance

Para validar a diferença de desempenho, foi executado um benchmark com 12.000.000 de iterações, variando a quantidade de workers. O objetivo não foi obter o valor exato de Pi, mas medir o impacto da divisão da carga de trabalho sobre o tempo total de execução.

Os resultados mostram que a execução com 4 workers apresentou o melhor tempo no ambiente de teste. A execução com 8 workers não obteve ganho adicional porque o custo de criação de processos, coordenação e disputa por CPU passou a superar o benefício da divisão. Esse comportamento é comum em sistemas paralelos: existe um ponto ótimo de paralelismo, e aumentar workers indefinidamente não garante melhoria linear.

Workers	Tempo (s)	Pi estimado	Speedup vs. 1 worker
1	4,359	3,142455	1,00x
2	2,410	3,141800	1,81x
4	1,691	3,142678	2,58x
8	2,166	3,142552	2,01x

Figura 2 - Gráfico comparativo de tempo de execução



7. Implantação e uso da aplicação

A aplicação foi preparada para execução por Docker Compose. Esse formato atende ao requisito de demonstrar a topologia distribuída com múltiplos containers, além de facilitar o envio ao professor e a reprodução do ambiente. Para executar, basta acessar a pasta do projeto e rodar o comando `docker compose up --build`.

Após a inicialização, o usuário pode acessar o front-end em <http://localhost:8501>. A documentação interativa da API fica disponível em <http://localhost:8000/docs>. Na interface, o usuário informa a quantidade de iterações, define o número de workers/chunks e inicia o processamento. Ao final, a tela exibe a estimativa de Pi, o tempo gasto e os detalhes retornados pela API.

Para deploy em nuvem, recomenda-se publicar o código no GitHub e utilizar uma plataforma compatível com Docker. Em Railway, Render ou Fly.io, os serviços devem ser configurados separadamente: API, front-end, Redis e workers. Caso a plataforma limite múltiplos serviços no plano gratuito, a entrega pode ser complementada com evidências do ambiente Docker Compose em funcionamento, conforme previsto no enunciado da atividade.

8. Entregáveis acadêmicos

- **Código-fonte:** Disponibilizar o projeto no GitHub, mantendo as pastas backend, frontend, scripts e o arquivo `docker-compose.yml`.
- **Link da aplicação:** Publicar a interface em uma plataforma de nuvem ou apresentar evidências claras do ambiente Docker Compose em execução.
- **Vídeo demonstrativo:** Gravar de 2 a 4 minutos mostrando a interface, execução com 1 worker e execução com 4 workers, destacando a diferença de tempo.
- **Relatório técnico:** Submeter este relatório em PDF, contendo arquitetura, lógica de paralelismo, lógica de distribuição e comparação de performance.

9. Roteiro sugerido para o vídeo de demonstração

8. Apresentar rapidamente o objetivo da aplicação e a arquitetura utilizada.
9. Mostrar o Docker Compose com Redis, API, front-end e quatro workers ativos.
10. Acessar a interface Streamlit e executar o cálculo com 1 worker.
11. Repetir o teste com 4 workers usando a mesma quantidade de iterações.
12. Comparar os tempos exibidos e explicar que a carga foi dividida em chunks independentes.
13. Finalizar mostrando o repositório GitHub e a estrutura do código-fonte.

10. Conclusão

O projeto desenvolvido atende aos objetivos da atividade ao demonstrar, em uma aplicação prática, a integração entre processamento paralelo, processamento distribuído e computação em nuvem. O problema escolhido, cálculo de Pi por Monte Carlo, permitiu dividir a carga de trabalho em tarefas independentes, tornando clara a aplicação dos conceitos de workers, filas, containers e consolidação de resultados.

Os testes indicaram redução expressiva no tempo de execução quando a carga foi distribuída entre múltiplos workers. A execução com 4 workers apresentou speedup de aproximadamente 2,58 vezes em relação à execução com 1 worker. Embora o ganho não seja perfeitamente linear, o resultado é coerente com limitações reais de infraestrutura, como overhead de orquestração, comunicação e disponibilidade de CPU.

Dessa forma, a aplicação evidencia os benefícios e também os limites do paralelismo em sistemas distribuídos. A arquitetura proposta é simples, reproduzível e extensível, podendo ser adaptada para outros problemas de maior escala, como processamento de imagens, análise textual ou cálculo estatístico sobre grandes datasets.

Referências

Docker. Docker Compose Documentation. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 08 jun. 2026.

Celery Project. Celery - Distributed Task Queue. Disponível em: <https://docs.celeryq.dev/>. Acesso em: 08 jun. 2026.

FastAPI. FastAPI Documentation. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 08 jun. 2026.

Streamlit. Streamlit Documentation. Disponível em: <https://docs.streamlit.io/>. Acesso em: 08 jun. 2026.

Redis. Redis Documentation. Disponível em: <https://redis.io/docs/>. Acesso em: 08 jun. 2026.